

LAB SESSION 9

Lab Name: Coding, Implementation & Version Control

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Implement system modules following design specifications
 - Apply clean coding practices and TypeScript standards
 - Use Git for version control and collaborative development
-

Theory

Implementation must:

- **Follow design:** Code must reflect the architecture and UML models defined in Lab Session 6
 - **Follow coding standards:** TypeScript strict typing, consistent naming conventions, modular structure
 - **Use version control:** Git enables tracking changes, collaboration, and rollback capability
-

Task 1: Implemented Modules

Module 1: Authentication System

Files Implemented:

- `src/lib/auth.ts` — NextAuth configuration with credentials, Google, GitHub providers
- `src/lib/db.ts` — Prisma client singleton with pg adapter and SSL configuration
- `src/app/api/register/route.ts` — User registration API
- `src/app/api/verify-email/route.ts` — Email verification API
- `src/app/api/resend-verification/route.ts` — Resend verification email API
- `src/app/api/auth/2fa/send/route.ts` — 2FA OTP send API
- `src/app/api/auth/2fa/verify/route.ts` — 2FA OTP verify API
- `src/app/login/page.tsx` — Login page with credentials and OAuth
- `src/app/register/page.tsx` — Registration page
- `src/app/verify-2fa/page.tsx` — 2FA OTP verification page

Key Implementation — 2FA Send OTP (`src/app/api/auth/2fa/send/route.ts`):

```
function generateOTP(): string {
  return Math.floor(100000 + Math.random() * 900000).toString();
}

export async function POST(req: NextRequest) {
  const { email } = await req.json();
```

```
const user = await db.user.findUnique({
  where: { email },
  select: { id: true, email: true, name: true, twoFactorEnabled: true },
});

if (!user || !user.twoFactorEnabled) {
  return NextResponse.json({ message: "2FA not enabled" }, { status: 400 });
}

const otp = generateOTP();
const expires = new Date(Date.now() + 10 * 60 * 1000); // 10 minutes

await db.user.update({
  where: { email },
  data: { twoFactorCode: otp, twoFactorExpires: expires },
});

// Send OTP via Nodemailer
await transporter.sendMail({ to: email, subject: "Your 2FA Code", html:
`...` });
}
```

Key Implementation — Auth.ts 2FA Check:

```
async authorize(credentials) {
  const user = await db.user.findUnique({ where: { email: credentials.email
} });

  if (!user.emailVerified) throw new Error("Please verify your email before
logging in");

  // 2FA bypass verification
  if (credentials.twoFactorVerified === "true" && credentials.password ===
"__2fa_verified__") {
    if (user.twoFactorCode !== null) throw new Error("Invalid
credentials");
    return { id: user.id, email: user.email, name: user.name, role:
user.role };
  }

  const isPasswordMatch = await bcrypt.compare(credentials.password,
user.password);
  if (!isPasswordMatch) throw new Error("Incorrect password");

  if (user.twoFactorEnabled) throw new Error("2FA_REQUIRED:" + user.email);

  return { id: user.id, email: user.email, name: user.name, role: user.role
};
}
```

Module 2: Appointment System

Files Implemented:

- `src/app/api/appointment/route.ts` — GET (fetch appointments), POST (create appointment)
- `src/app/api/appointment/[id]/route.ts` — PATCH (update status), DELETE (delete appointment)
- `src/components/BookAppointment.tsx` — Patient appointment booking UI
- `src/components/MyAppointments.tsx` — Patient appointments list UI
- `src/components/DoctorAppointments.tsx` — Doctor appointments management UI

Key Implementation — Create Appointment:

```
export async function POST(req: NextRequest) {
  const session = await getServerSession(authOptions);
  if (!session) return NextResponse.json({ message: "Unauthorized" }, {
status: 401 });

  const { doctorId, date, time, reason } = await req.json();

  const patientProfile = await db.patientProfile.findUnique({
    where: { userId: session.user.id },
  });
}
```

```
const appointment = await db.appointment.create({
  data: {
    doctorId,
    patientId: patientProfile.id,
    date: new Date(date),
    time,
    reason,
    status: "PENDING",
  },
});

return NextResponse.json({ appointment }, { status: 201 });
}
```

Module 3: Prescription System

Files Implemented:

- `src/app/api/prescription/route.ts` — POST (create prescription), GET (fetch prescriptions)
- `src/components/PrescriptionForm.tsx` — Doctor prescription issuance UI
- `src/components/DownloadPDF.tsx` — PDF generation using html2canvas + jsPDF

Key Implementation — PDF Download:

```
const handleDownload = async () => {
  const element =
document.getElementById(`prescription-${prescription.id}`);
  const canvas = await html2canvas(element);
  const imgData = canvas.toDataURL("image/png");
  const pdf = new jsPDF();
  pdf.addImage(imgData, "PNG", 0, 0, 210, 297);
  pdf.save(`prescription-${prescription.id}.pdf`);
};
```

Module 4: Medicine Reminder System

Files Implemented:

- `src/app/api/medicine-reminder/route.ts` — GET, POST reminders
- `src/app/api/medicine-reminder/[id]/route.ts` — PATCH (mark taken), DELETE
- `src/app/api/medicine-reminder/send-notifications/route.ts` — Cron job endpoint

Key Implementation — Send Notifications (Cron):

```
export async function POST(req: NextRequest) {
  const apiKey = req.headers.get("x-api-key");
```

```
if (apiKey !== process.env.CRON_API_KEY) {
  return NextResponse.json({ message: "Unauthorized" }, { status: 401 });
}

const today = new Date();
const reminders = await db.medicineReminder.findMany({
  where: {
    taken: false,
    startDate: { lte: today },
    endDate: { gte: today },
  },
  include: { patient: { include: { user: true } } },
});

// Send email for each due reminder
for (const reminder of reminders) {
  await transporter.sendMail({ to: reminder.patient.user.email, ... });
}
```

Module 5: Medical Vault

Files Implemented:

- `src/app/api/vault/route.ts` — POST (upload document)
- `src/app/api/vault/[id]/route.ts` — DELETE (delete document)
- `src/components/FileUpload.tsx` — File upload UI with Supabase Storage integration

Key Implementation — File Upload:

```
export async function POST(req: NextRequest) {
  const session = await getServerSession(authOptions);
  const formData = await req.formData();
  const file = formData.get("file") as File;

  // Upload to Supabase Storage
  const { data, error } = await supabase.storage
    .from("medical-reports")
    .upload(`${session.user.id}/${file.name}`, file);

  if (error) return NextResponse.json({ message: "Upload failed" }, {
    status: 500 });

  const { data: urlData } = supabase.storage
    .from("medical-reports")
    .getPublicUrl(data.path);

  await db.medicalVault.create({
    data: { fileName: file.name, fileUrl: urlData.publicUrl, patientId:
      patientProfile.id },
```

```
});  
}
```

Module 6: Admin Dashboard

Files Implemented:

- `src/app/api/admin/stats/route.ts` — Real-time statistics
- `src/app/api/admin/users/route.ts` — User management
- `src/app/api/admin/users/[id]/route.ts` — Delete user
- `src/app/api/admin/appointments/route.ts` — All appointments
- `src/app/api/admin/contacts/route.ts` — Contact messages
- `src/components/AdminDashboard.tsx` — Admin dashboard UI

Key Implementation — Admin Stats:

```
export async function GET() {  
  const session = await getServerSession(authOptions);  
  if (session?.user?.role !== "ADMIN") {  
    return NextResponse.json({ message: "Forbidden" }, { status: 403 });  
  }  
  
  const [users, patients, doctors, appointments, prescriptions, contacts] =  
    await Promise.all([  
      db.user.count(),  
      db.patientProfile.count(),  
      db.doctorProfile.count(),  
      db.appointment.count(),  
      db.prescription.count(),  
      db.contactMessage.count(),  
    ]);  
  
  return NextResponse.json({ users, patients, doctors, appointments,  
    prescriptions, contacts });  
}
```

Module 7: Settings

Files Implemented:

- `src/app/api/settings/profile/route.ts` — Update profile name
- `src/app/api/settings/password/route.ts` — Change password
- `src/app/api/settings/2fa/route.ts` — Toggle 2FA
- `src/components/SettingsForm.tsx` — Settings UI with profile, password, and 2FA sections

Task 2: Git Version Control

Repository

- **GitHub Repository:** <https://github.com/tusharsno/mediscript-e>
- **Branch:** `main`
- **Deployment:** Vercel (auto-deploy on push to `main`)

Git Workflow Used

1. Feature development on local machine
2. `git add .` – Stage all changes
3. `git commit -m "descriptive message"` – Commit with clear message
4. `git push origin main` – Push to GitHub
5. Vercel auto-deploys from GitHub

Key Commits

Commit Message	Description
Initial project setup	Next.js 16 project initialization with Prisma, NextAuth
Add authentication system	Registration, login, email verification
Add appointment booking	Patient and doctor appointment management
Add prescription system	Doctor prescription issuance and PDF download
Add medicine reminders	Reminder scheduling with cron job
Add medical vault	Supabase Storage file upload
Add admin dashboard	Real-time stats and user management
Fix Vercel deployment - Prisma 7 config	Production deployment fix
Fix SSL with sslmode=no-verify	Database SSL connection fix
Add 2FA email OTP verification	Two-factor authentication feature
Add SecuritySection to landing page	Security section UI

.gitignore Configuration

```
.env
.env.local
.env.production
node_modules/
.next/
.vercel/
```

Task 3: Secure Coding Practices Applied

Practice	Implementation
Input validation	All API routes validate required fields before processing
Parameterized queries	Prisma ORM used for all database operations — no raw SQL
Password hashing	<code>bcrypt.hash(password, 10)</code> — never stored in plaintext
Environment variables	All secrets in <code>.env</code> — never hardcoded in source code
Role-based authorization	<code>getServerSession()</code> checked on every protected API route
TypeScript strict typing	All components and API routes use TypeScript interfaces
Selective data queries	Prisma <code>select</code> used to return only required fields
Token expiry	Verification tokens (24h), OTP (10min), JWT sessions (30 days)
Error handling	Try-catch blocks on all async operations
SSL/TLS	Database connection uses <code>sslmode=no-verify</code> for Supabase

README File

MediScript-E – Digital Healthcare Platform

A modern, secure digital healthcare platform built with Next.js 16.

Tech Stack

- Frontend: Next.js 16, React 19, Tailwind CSS 4, Framer Motion
- Backend: Next.js API Routes, NextAuth.js, Prisma 7
- Database: PostgreSQL (Supabase)
- Storage: Supabase Storage
- Email: Nodemailer (Gmail SMTP)
- Deployment: Vercel

Features

- Email/Password + OAuth (Google, GitHub) authentication
- Email verification + Two-Factor Authentication (2FA)
- Appointment booking and management
- Digital prescriptions with PDF download
- Medicine reminders with automated email alerts
- Medical vault for secure document storage
- Admin dashboard with real-time statistics

Setup

1. Clone: `git clone https://github.com/tusharsno/mediscript-e.git`
2. Install: `pnpm install`
3. Configure: Copy `.env.example` to `.env` and fill in values
4. Generate Prisma client: `npx prisma generate`
5. Run: `pnpm dev`


```
## Live Demo  
https://mediscript-e.vercel.app
```

Key Findings / Learning Outcomes

- Successfully implemented **7 major modules** with **30+ API endpoints** following the design from Lab Session 6
- Learned that **TypeScript strict typing** catches bugs at compile time, reducing runtime errors
- Understood the importance of **Git commit messages** — clear messages make project history readable
- Applied **secure coding practices** throughout: input validation, parameterized queries, environment variables
- Recognized that **Prisma ORM** significantly reduces boilerplate code while ensuring type-safe database operations
- Learned that **serverless architecture** (Vercel) requires careful consideration of connection pooling and cold starts
- Version control enabled safe experimentation — broken changes could be reverted using `git revert`